

AIPLA — Internal Hosting Requirements

Resources and questions for University of Copenhagen IT

AIPLA project team

2026-06-18

Table of contents

1. Purpose	1
2. What the application is (in brief)	2
3. Data residency and privacy (why hosting at UCPH is attractive)	2
4. Hosting options	2
5. Resource list (what UCPH would provide)	3
6. AI models and GPU sizing (the main open decision)	4
Responsiveness and capacity	5
7. Questions for UCPH IT	6
8. What we will hand over	6

Draft for review

This is a draft prepared for the University of Copenhagen IT department. Alongside it we provide infrastructure-as-code (Terraform) and operational runbooks so your team can stand the system up against the resources below, or use them as a costing reference for your own deployment.

1. Purpose

AIPLA (AI in Physics Learning and Assessment) currently runs on Google Cloud in the EU (Finland region). As part of the handover we are scoping what it would take to host the application **inside University of Copenhagen infrastructure** — either fully on-premises or in a hybrid arrangement.

This document lists the resources the application needs so your team can assess feasibility and estimate hardware and hosting effort. It ends with a short set of questions we need answered to firm up the plan.

The application is built on portable, open standards — Linux containers, PostgreSQL, S3-compatible object storage, OpenID Connect single sign-on, and open-source model inference. Migrating it is therefore a matter of providing equivalent infrastructure, **not** rewriting the software.

2. What the application is (in brief)

Component	What it does	Standard technology
Web application	Student chat interface + teacher dashboard	Linux containers (Next.js + Python)
Database	Application data + document-search index	PostgreSQL (with the pgvector extension)
Object storage	Uploaded documents, optional lesson audio, caches	S3-compatible (e.g. MinIO) or NFS
AI model inference	The language models behind the tutor	Open-weight models served on GPUs (or cloud API)
Authentication	Teacher sign-in; anonymous student join	Institutional SSO (OIDC/SAML) + anonymous codes
Observability	Logs, metrics, traces (kept internal)	OpenTelemetry → Grafana/Prometheus or ELK

The web application and database are modest in size; the **GPU for model inference is the main new hardware consideration**, and is covered in sections 5–6.

3. Data residency and privacy (why hosting at UCPH is attractive)

This is likely the strongest reason to host internally:

- **No personally identifying student data is collected — by design.** Students join a lesson by entering an anonymous group code on a shared device. The system stores nothing that identifies an individual student: no names, no emails, no device identifiers. The “personal data” category is empty by construction, which substantially simplifies data-protection review.
- **Teachers** sign in with institutional credentials (single sign-on).
- **Today** all data is held in the EU (Google Cloud, Finland). **Hosting at UCPH** would place all data — including any optional, consent-gated lesson recordings — entirely on university infrastructure: the strongest possible data-sovereignty posture.

4. Hosting options

There is a spectrum between today’s full-cloud setup and a fully on-premises deployment. UCPH does not have to choose the extreme.

	A. Full cloud (today)	B. Inference on-prem	C. App + data on-prem (recommended)	D. Fully on-prem
AI models	Cloud API	UCPH GPUs serve most; cloud for overflow	UCPH GPUs serve most; cloud API only for the hardest tasks	All on UCPH GPUs
App + database + storage	Cloud (EU)	Cloud (EU)	UCPH	UCPH
Student data leaves UCPH?	Stays in EU cloud	Stays in EU cloud	No (except a few model prompts)	No
GPU hardware needed	None	High	Medium (single high-end GPU)	High (GPU cluster)
Effort	—	Low–medium	Medium	High

Our recommendation is C: the application and all student data live at UCPH, while a small set of the most demanding AI tasks may still use an EU cloud model API until on-premises models are proven equal. **B** is a sensible stepping stone if GPUs arrive before the rest of the migration. **D** is the end state once a GPU cluster is in place.

5. Resource list (what UCPH would provide)

Quantities depend on the hosting option chosen (section 4). Figures are for pilot scale (roughly 10 teachers and their classes) and are order-of-magnitude for planning.

Compute (application — modest):

- Container hosting: a Kubernetes namespace **or** 1–2 virtual machines running Docker. Approximately **4 vCPU / 8 GB RAM total** for the web app, backend, and a small document-parsing service.
- A container image registry and a CI runner for builds.

GPU (AI model inference — the main hardware decision; see section 6):

- **Option C (recommended):** one high-end GPU (e.g. **NVIDIA H100 80 GB**) for the main model — comfortably runs a current single-GPU open-weight model such as Qwen 3.6 (27–35B)¹ — plus one smaller GPU (e.g. **L4 / A10, 24 GB**) for text-search embeddings and speech-to-text.

¹Qwen 3.6 sizes and VRAM (27B dense and 35B-A3B MoE; ~17–21 GB at 4-bit, fits a single 24 GB GPU): *Qwen 3.6 VRAM & Hardware Requirements*, Will It Run AI (2026) — <https://willitrunai.com/blog/qwen-3-6-vram-requirements>; *Qwen3.6 35B-A3B: Specifications and GPU VRAM*, apxml — <https://apxml.com/models/qwen36-35b-a3b>.

- **Option D (full sovereignty):** a cluster of $8 \times \text{H200}$ to serve a frontier-class open-weight model (e.g. GLM 5.2, ~744B parameters, at FP8 — roughly 1.1 TB aggregate VRAM)², plus the smaller embeddings / speech GPU.
- Inference is served with standard open-source software (vLLM or Ollama).

Database:

- **PostgreSQL 15+ with the pgvector extension** — serves application data, document-search vectors, session state, and analytics. Managed Postgres if available; otherwise a standard (ideally highly-available) instance.

Object storage:

- **S3-compatible storage (e.g. MinIO) or NFS** — uploaded documents, optional lesson recordings, caches. Tens of GB at pilot scale; audio dominates if recording is enabled.

Identity:

- **Teacher single sign-on** from the UCPH identity provider (OpenID Connect or SAML). Students require **no** identity provider (anonymous codes only).

Supporting services:

- A secrets store (e.g. HashiCorp Vault or Kubernetes secrets).
- A logging / metrics stack (Grafana + Prometheus / Loki, or ELK).
- Ingress with TLS certificates; a policy decision on whether any external model API calls are permitted (relevant only for hybrid options B / C).

6. AI models and GPU sizing (the main open decision)

During the pilot the application uses top-tier commercial models for quality. For self-hosting, these are replaced by **open-weight models** run on UCPH GPUs. The model choice is being validated by an ongoing capability evaluation against real physics-tutoring tasks; the **hardware to provision depends on which hosting option (section 4) UCPH chooses**:

Tier	Example open-weight model	Hardware	Suitable for
Single GPU (Option C floor)	Qwen 3.6 (27B dense, or 35B-A3B MoE)	1 $\times$ H100 / A100 80 GB (runs at 4-bit on a 24 GB GPU)	Tutor chat, document tasks; the realistic starting point

²GLM 5.2 architecture and VRAM (~744B total / 39B active MoE, 1M-token context, MIT-licensed; ~744 GB at FP8, fits one $8 \times \text{H200}$ node): *GLM-5.2 vLLM recipe* — <https://recipes.vllm.ai/zai-org/GLM-5.2>; *Self-Host GLM 5.2: Hardware, vLLM Setup, and Cost*, ofox.ai (2026) — <https://ofox.ai/blog/glm-5-2-self-host-vllm-hardware-cost-2026/>; *GLM-5.2 1M context on a single 8xH200 node*, Phala — <https://phala.com/posts/glm-5-2-1m-context-8xh200>.

Tier	Example open-weight model	Hardware	Suitable for
GPU cluster (Option D)	GLM 5.2 (~744B params / 39B active MoE, 1M context, MIT-licensed — currently the strongest open-weight model)	8 × H200 at FP8 (~1.1 TB aggregate VRAM); BF16 needs more	All tasks including the hardest reasoning; whole-institution scale
Supporting GPU	Embeddings + speech-to-text (Whisper)	1 × 24 GB GPU (or CPU for embeddings)	Document search + voice

We are happy to walk your team through the evaluation results and recommend a specific configuration once your GPU availability is known.

Responsiveness and capacity

Two figures matter, and they answer different questions:

- **Per-student streaming speed** — how fast a single reply appears. Human reading is roughly 5–10 tokens per second, so anything above ~20–30 tokens per second per student already feels immediate. Both recommended setups exceed this comfortably for an individual user.
- **Aggregate throughput** — the total tokens per second served across *all* students at once (the serving software batches requests continuously). This is the capacity figure.

Recent published benchmarks put a single H100 serving a Qwen-3-class model at roughly **2,000+ tokens/second aggregate**³, and an 8 × H200 cluster serving GLM 5.2 at about **1,200 tokens/second**⁴ (with speculative decoding). In practice that means a single H100 comfortably serves a full classroom concurrently, while each student’s reply still streams faster than they can read.

Setup	Aggregate throughput	Per-student feel
Option C — Qwen 3.6 on 1 × H100	~2,000+ tokens/s	tens of tokens/s (well above reading speed)
Option D — GLM 5.2 on 8 × H200 (FP8)	~1,200 tokens/s	~30–60 tokens/s

Making responses faster (no extra hardware). The largest gains for a tutoring workload come from software, not silicon:

³Qwen-3-class throughput on a single H100 (~2,352 tokens/s aggregate via vLLM): *Optimizing Qwen3-32B Throughput on NVIDIA H100*, GPUStack — <https://docs.gpustack.ai/2.0/performance-lab/qwen3-32b/h100/>; day-one cost/throughput comparison for Qwen3.6-27B: *Qwen3.6-27B bakeoff*, LLMKube — <https://llmkube.com/blog/qwen3-6-27b-bakeoff>.

⁴GLM 5.2 throughput on 8 × H200 (~1,215 tokens/s sustained output, FP8, SGLang + EAGLE speculative decoding; ~1,370 tokens/s on 8 × B200): *GLM-5-FP8 Inference Benchmark: 8×B200 vs 8×H200* — <https://gist.github.com/BenHamm/dcd09f595fef141567a39582f502cef4>.

- **Tiered model routing** — most tutoring turns are handled by the smaller, faster model (Qwen 3.6), reserving the large model for genuinely hard reasoning. The typical reply is then both faster and cheaper.
- **Prefix caching** — the tutor’s fixed instructions and lesson context are identical every turn, so they are processed once and cached, sharply reducing the time to the first word of a reply.
- **Speculative decoding** (e.g. EAGLE) — 1.5–3× faster reply streaming; the GLM 5.2 figure above already uses it.
- **FP8 precision** — already assumed in the sizing above.

Newer or additional hardware (e.g. B200) raises both numbers further, but is only needed if concurrency or large-model latency must improve beyond the above.

7. Questions for UCPH IT

To turn this into a concrete plan and estimate, we need to know:

1. **GPU hosting.** Do you have, or can you provision, GPU hosting for model inference? A single high-end GPU (Option C) or a cluster (Option D)? What specification and on what timeline?
2. **Containers and database.** Do you offer Kubernetes or VM hosting, and a managed **PostgreSQL** (ideally with the **pgvector** extension)? Is S3-compatible object storage available?
3. **Single sign-on.** Which protocol does the UCPH identity provider support for application integration — OpenID Connect or SAML?
4. **Data-protection precedent.** Is there an existing pattern for student-facing services that has already cleared your data-protection review, which we could align with rather than starting from scratch?
5. **External connectivity (hybrid only).** For a hybrid setup, is it acceptable for the application to call an **EU-hosted** commercial model API for a small set of the hardest tasks (sending only the prompt, no student identity), or must the deployment be fully self-contained?

8. What we will hand over

- This requirements document and accompanying migration notes (the scoping half of the handover).
- **Infrastructure-as-code (Terraform)** plus operational runbooks, so your team can stand the system up against the resources above — or read them as a costing reference for your own deployment.
- A walkthrough session with your team to answer questions and agree the target configuration.

We can begin the migration when GPU / hosting availability is confirmed and the on-premises models clear our capability evaluation for the tasks they would take on; until then the system continues to run on EU cloud with no disruption.